LLM Reasoning in POMDP Problems

Daniel Zhu
Computer Science
University of Colorado Boulder
Boulder, USA

I. Introduction

When a robot starts up, whether by a system update, battery swap, etc, re-localization is a important part of getting back to full usefulness. However, standard methods like particle filters require dense maps and a lot of compute. Additionally, these localization methods often struggle to immediately determine a position when placed in sparse information environments, such as hospitals or factories with many similar-looking hallways. In such environments, relying solely on immediate surrounding information is insufficient. To successfully localize, a robot must take exploratory actions. For instance, while observing a single trashcan might not be enough to localize within a massive building, actively exploring to find a sequence of a trashcan, a door, and a sink can provide a unique feature signature. A critical component of this challenge is deciding where the robot should explore next to gather information most efficiently.

Large Language Models (LLMs) have been used time and time again to provide a "common sense" reasoning for many tasks. As such I want to evaluate the LLMs ability as a policy for this exploratory action.

So, in this project I'll be seeing how LLMs perform in solving POMDP problems like the Tiger Problem (as a proof of concept) and testing its use in the above problem when formulated as a POMDP (which i'll refer to as Explore-POMDP) this work aims to demonstrate that an LLM's general understanding and decision-making capabilities can serve as a viable alternative to manually propagating region-matching algorithms.

II. Background and Related Work

Recently, Large Language Models (LLMs) have been used repeatedly to provide "common sense" reasoning across various domains. The integration of LLMs with POMDPs and spatial navigation is an emerging area of research. For example, recent frameworks like CoCo-TAMP [1] have demonstrated that LLMs can provide common-sense priors (such as object co-location cues) to guide belief-space planning in partially observable environments. Similarly, research into LLM-guided probabilistic program induction [2] highlights how language models can serve as priors for estimating POMDP transition and observation functions.

In the realm of physical navigation, systems such as osmAG-LLM [3] and RoboAuditor-LLM [4] validate that LLMs can effectively utilize textual semantic maps to parse spatial relationships and plan executable, step-by-step exploration actions in unknown or sparse environments. Furthermore, the broader application of treating unobservable variables as a POMDP to be solved via LLM interaction has been formalized in frameworks like PAMDP [5], underscoring the mathematical viability of our approach.

III. Problem Formulation

Since the Tiger Problem is a famous POMDP problem, I'll briefly go over my formulation:

- **States (S):** The position of the tiger.

$$S = \{s_{left}, s_{right}\}$$

- **Actions (A):**

$$A = \{a_{listen}, a_{open\_left}, a_{open\_right}, a_{wait}\}$$

- **Transition Function (T):** $P(s'|s,a)$
  - If $a = a_{listen}$ or $a = a_{wait}$, the state remains the same: $s' = s$.
  - If $a \in \{a_{open\_left}, a_{open\_right}\}$, the episode typically resets, and the tiger is repositioned uniformly: $P(s'|s,a) = 0.5$ for all $s' \in S$.
- **Reward Function (R):** $R(s,a)$
  - $R(s, a_{listen}) = -1$
  - $R(s, a_{wait}) = 0$
  - $R(s_{left}, a_{open\_left}) = -100$ (Tiger found)
  - $R(s_{right}, a_{open\_right}) = -100$ (Tiger found)
  - $R(s_{left}, a_{open\_right}) = +10$ (Treasure found)
  - $R(s_{right}, a_{open\_left}) = +10$ (Treasure found)
- **Observations (Ω):**

$$\Omega = \{o_{left}, o_{right}, o_{null}\}$$

- **Observation Function (O):** $P(o|s',a)$
  - For $a = a_{listen}$, the sensor is noisy:

$$P(o_{left}|s_{left}, a_{listen}) = 0.85, \quad P(o_{right}|s_{left}, a_{listen}) = 0.15$$

  - For $a \in \{a_{wait}, a_{open\_left}, a_{open\_right}\}$, the observation is uninformative:

$$P(o_{null}|s,a) = 1.0$$

- **Discount Factor (γ):** 1.
- **Initial Belief (b_0):**

$$b_0 = [P(s_{left}) = 0.5, P(s_{right}) = 0.5]$$

Now to test an LLM's ability to localize and intelligently explore a unknown environment, I set up this problem as a POMDP where

- **States (S):** The true position of the "robot"
- **Actions (A):** At each turn, the llm can either move any of the cardinal directions, or if it's confident enough can guess its location.

  $A = \{a_{move\_N}, a_{move\_E}, a_{move\_S}, a_{move\_S}, a_{localize(x,y)}\}$
- **Transition Function (T):** We'll assume the robot always moves to the correct location.
- **Reward Function (R):** $R(s, a)$
  - $R(s, a_{move\_X}) = -1$
  - $R(s_{x,y}, a_{localize(x,y)}) = +100$ (Correct localization)
  - $R(s_{a,b}, a_{localize(x,y)}) = -100$ (Incorrect localization)
- **Observations (Ω):** The robot may only see the items at its current location and the available path ways it can move.
- **Observation Function (O):** We assume the robots sensors are good enough to always sense correctly.
- **Discount Factor (γ):** 1.
- **Initial Belief (b_0):** Evenly spread across the grid.

The reason why I did not include random orientation and observation noise is that in practice, it is really easy to have a compass and strong vision detection systems and I wanted to focus on the use case for my future project.

IV. Approach

For the Tiger POMDP I used Google's Gemini API as I don't have access to good compute. I used Google's new Gemma4 models (31b, 21b, e4b) as they had a high allowance of tokens and requests on their free tier.

I gave the following prompts to the LLMs to inform what it has to do:

System Instruction:

- You are an agent solving the Tiger Problem. Behind one door is a tiger (-100); behind the other is gold ($+10$). Listen: Cost -1, 85% accuracy.
  Open: Ends the episode.
  Wait: Cost 0.
  Output Format: Provide step-by-step reasoning, then: [ACTION: <action>]

Observation: Listen

- You listened and heard a tiger roar from the {direction}. What is your next action?

.

I recorded 25 conversations per model which includes their accumulated reward, episode length, and specific comments for me to anlayze myself.

For the ExplorePOMDP, I used only Gemini's 31B model as I felt it had the highest chance for success in this scenario. For the environment, I went with a simple 2d grid world where each node has a random chance of containing nothing, a desk, a plant, or a trashcan. I only limited to these simple objects to mimic what you could expect in a sparse environment.

For the prompts, I roughly used:

System Instruction:

- You are a robot localizing yourself in a known 2D grid map. Your starting coordinate is completely unknown. You have a built-in compass and know your absolute orientation at all times.
  1. Every turn, update a Candidate List of possible states formatted as (x,y).
  2. Explicitly write out your logic for eliminating coordinates. Example: "I moved N. If I was at (1,1), I am now at (1,2). (1,2) has a Desk. My observation says 'None'. I eliminate (1,1) from my original list."
  3. Your job is to reduce the maximum amount of uncertainty each turn. Therefore your movement decisions should be based on which paths will eliminate the most candidates in the next observation. Explain your reasoning step-by-step each turn. 3. When exactly 1 candidate remains, output: [ACTION: localize(x,y)]
  4. Output your action strictly on a new line at the very end in this format: [ACTION: <your action>].

Observation

- "item_here": "example_item"
- "paths_available": ["N", "E", "S", "W"]

To ensure reproducibility of the experiments on the Tiger and ExplorePOMDP environments, the full source code, including the evaluation scripts and model configuration parameters, and specific messages, is hosted on GitHub[1].

V. Results

To start off with the Tiger POMDP Problem, here are my results over 25 simulations of the problem.

TABLE I
Comparative Performance of Gemma4 Model Variants on Tiger Problem POMDP Over 25 Episodes

Model Variant	Average Reward	Avg. Episode Length
Gemma4 e4b	-27.20	3.12
Gemma4 26b	-0.88	2.68
Gemma4 31b	$\mathbf{7.20}$	3.80

For all models in general, they all succesfully reasoned thorugh math that their best option is to listen first, but they diverged on what to do next.

For the e4b model, one of their more lighter models with only 8b parameters, I see often that it then ignores the benefit of information gain with the listen option and just compares the relative rewards of the left and right doors. For example, say we hear roar from the left, it would calculate expected value for left as $0.85 * (-100) + 0.15 * (10) = -83.5$ and for right as $0.15 * (-100) + 0.85 * (10) = -6.5$, and as such chooses right. Sometimes it gets the math wrong or doens't do

[1]https://github.com/dizyweather/LLM_POMDP

any math and just uses the roar as a likely truth like a QMDP would.

Comparatively, for the 21B model, it does better as it started to consider the value of listening again but sometimes miscalculates the EV of listening again by adding the previous reward from listening and adding it like a sunk cost, and assumes that it will have to pick a door after if it does listen again. Around half the time it does the above, but the other half it properly calculates the EV and waits for a net two listens in one direction before guessing.

Finally, the 31B never ran into the above two problems and always properly keeps listening until it has a net two listens in one direction.

Whats exciting is when nudging the model to use the 'wait' option with "we should wait right?" the LLM's common sense reasoning shows through.

"Regarding the suggestion to 'wait': waiting has no cost, but it also provides no new information and does not change the probabilities. It would simply delay the decision without improving the expected outcome. Therefore, the most rational action to maximize my reward is to listen again to further reduce the risk of hitting the tiger.[ACTION: listen]"

Even though the reward of 'wait' is less than 'listen', the LLM properly identified that this action does not gain us information and would simply just delay the decision, and thus never chose the option. It doesn't fall into the same pitfalls as an alpha vector approach.

For the ExplorePOMDP there isn't much to gain out of just looking at reward, but to examine the success rate and LLM's train of thought.

Grid Size	Success Rate
5x5	5/5
10x10	5/5
20x20	4/5
30x30	1/5

For the 10x10 and smaller maps, the LLM sucessfully localized in all situations, and reading the logs has a basic understanding of how to choose actions with high information gain. Paraphrasing what the llm said:

- move-N: All 16 candidates result in "plant". No information gained.
- move-S: Results in "none" for 10 candidates, "desk" for 3, "trashcan" for 1, and "plant" for 1. (Max group size: 10)
- move-E: Results in "none" for 8 candidates, "trashcan" for 4, and "desk" for 4. (Max group size: 8)
- move-W: Results in "none" for 9 candidates, "desk" for 3, "trashcan" for 2, and "plant" for 2. (Max group size: 9)

  The move-E action provides the most discrimination by minimizing the size of the largest resulting candidate group.

Although it did not choose the worse option of North, it did not pick the best option as the max information gain is actually to move West.

But for larger maps, we can see the LLM tripping up. For example trying to keep track of transitions it fumbled saying $(x+1, y+1) \rightarrow (x+1, y-1)$ when moving south. When in reality it should be $(x+1, y)$.

For the 30x30 map, many of them encountered an "Internal Error", so probably and error on the API side. The one that didn't encounter this error succeded.

VI. Conclusion and Future Work

In conclusion, for a simple POMDP problem such as Tiger POMDP, some current LLMs have been shown to be able to solve the problem entirely on its own without help. Additionally they show a good common understanding allowing them to avoid pitfalls such as just waiting.

For more complex problems like ExplorePOMDP, the Gemma4 31b model struggles to complete tasks for large arenas. Often making simple mistakes when propagating forward and not properly doing the best in information gain (which I think could improve with better prompting). Not mentioned in the results, trying to task the LLM on huge arenas proves difficult with the token limit and time it needs to parse through.

These point to that we could improve such faults if we handle parts of region matching and propagation manually while allowing the LLMs with their general understanding as the decision making portion a viable solution to my future project. While additionally reducing the amount of tokens and context space we take up.

LLMs, although not the best, provide a great deal of flexibility in a range of applications. Building around them by giving them the proper tools seem like a potential in my future project.

VII. Additional Details

All of the work was done by Daniel with the aid of LLMs. The author grants permission for this report to be posted publicly. All LLM models used were off the shelf, the open source Gemma 4 Models from Google.

The project scope did change from the proposal adding the evaluation of TigerPOMDP as our minimum working example as the original proposal was more optimistic of the time that we'd have for the project. Which was discussed with the professor.

But in the end, I feel like I accomplished a good portion of the main goal in evaluating these llms as a policy. DMU methods were helpful as baseline comparisons, but for the ExplorePOMDP formulation, I do feel it was not properly formulated as reward wasn't highly considered in the LLM's decision making. Additional thoughts about it is that it's hard to really give a reward or punishment to LLM based policies especially for long horizon tasks. The distinction between varying magnitudes of negative rewards (e.g., the cost of listening vs. the cost of a wrong guess) is often lost in token-level processing.

REFERENCES

[1] Anonymous: Large-language-model-guided state estimation for partially observable task and motion planning (2026)

[2] Curtis, A., Tang, H., Kaelbling, L.: Llm-guided probabilistic program induction for pomdp model estimation (2025)

[3] Xie, e.a.: osmag-llm: Zero-shot open-vocabulary object navigation via semantic maps and large language models reasoning. IEEE Xplore (2026)

[4] Cai, W., Huang, L., Zou, Z.: Roboauditor-llm: A robotic system for energy auditing via a large language model and an actionable semantic feature map. ASCE OPEN: Multidisciplinary Journal of Civil Engineering **3**(1) (2025)

[5] Yang, Z., Huang, Y., Chen, S., Wu, X., Yao, J., Feng, J.: Pamdp: Interact to persona alignment via a partially observable markov decision process. In: The Fourteenth International Conference on Learning Representations. (2026)